

说明

本次任务是在10.0.1.79服务器下新建用户ccj，使用第四张显卡进行

一、环境安装

1、安装Python

python服务器上可能本来就有，但是尽量不要使用这个，因为本身服务器共享，依赖可能会冲突。

所以我们会使用miniforge/anaconda来安装

```
# 安装miniforge
# 参考附录1
# 创建python环境，老师建议环境名称使用llamafactory_姓名缩写
conda create -n llamafactory_ccj python=3.11
# 创建完环境，需要激活环境
conda activate llamafactory_ccj
```

(1) 创建环境

```
ccj@user-SYS-4028GR-TR2-B0004:~$ source ~/.bashrc
(base) ccj@user-SYS-4028GR-TR2-B0004:~$ conda create -n llamafactory_ccj python=3.11
Retrieving notices: done
Channels:
 - conda-forge
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: /home/ccj/miniforge3/envs/llamafactory_ccj

added / updated specs:
  - python=3.11

The following packages will be downloaded:
```

package	build		
ca-certificates-2026.5.20	hbd8a1cb_0	127 KB	conda-forge
libexpat-2.8.1	hecca717_0	75 KB	conda-forge
libns1-2.0.1	hb9d3cd8_1	33 KB	conda-forge
libuuid-2.42.1	h5347b49_0	39 KB	conda-forge
libxcrypt-4.4.36	hd590300_1	98 KB	conda-forge
pip-26.1.2	pyh8b19718_0	1.1 MB	conda-forge
python-3.11.15	hd63d673_0_cpython	29.5 MB	conda-forge
wheel-0.47.0	pyhd8ed1ab_0	33 KB	conda-forge
Total:		31.1 MB	

```
The following NEW packages will be INSTALLED:
 _openmp_mutex      conda-forge/linux-64::_openmp_mutex-4.5-20_gnu
 bzip2              conda-forge/linux-64::bzip2-1.0.8-hda65f42_9
 ca-certificates    conda-forge/noarch::ca-certificates-2026.5.20-hbd8a1cb_0
```

```
package |-----| build
ca-certificates-2026.5.20 | hbd8a1cb_0 | 127 KB | conda-forge
libexpat-2.8.1 | hecca717_0 | 75 KB | conda-forge
libnsl-2.0.1 | hb9d3cd8_1 | 33 KB | conda-forge
libuuid-2.42.1 | h5347b49_0 | 39 KB | conda-forge
libxcrypt-4.4.36 | hd590300_1 | 98 KB | conda-forge
pip-26.1.2 | pyh8b19718_0 | 1.1 MB | conda-forge
python-3.11.15 | hd63d673_0_cpython | 29.5 MB | conda-forge
wheel-0.47.0 | pyhd8ed1ab_0 | 33 KB | conda-forge
-----|-----|
Total: 31.1 MB

The following NEW packages will be INSTALLED:

_openmp_mutex      conda-forge/linux-64::_openmp_mutex-4.5-20_gnu
bzip2              conda-forge/linux-64::bzip2-1.0.8-hda65f42_9
ca-certificates    conda-forge/noarch::ca-certificates-2026.5.20-hbd8a1cb_0
ld_impl_linux-64  conda-forge/linux-64::ld_impl_linux-64-2.45.1-default_hbd61a6d_102
libexpat           conda-forge/linux-64::libexpat-2.8.1-hecca717_0
libffi            conda-forge/linux-64::libffi-3.5.2-h3435931_0
libgcc            conda-forge/linux-64::libgcc-15.2.0-he0feb66_19
libgcc-ng         conda-forge/linux-64::libgcc-ng-15.2.0-h69a702a_19
libgomp           conda-forge/linux-64::libgomp-15.2.0-he0feb66_19
liblzma           conda-forge/linux-64::liblzma-5.8.3-hb03c661_0
libnsl            conda-forge/linux-64::libnsl-2.0.1-hb9d3cd8_1
libsqlite         conda-forge/linux-64::libsqlite-3.53.1-h0c1763c_0
libuuid           conda-forge/linux-64::libuuid-2.42.1-h5347b49_0
libxcrypt         conda-forge/linux-64::libxcrypt-4.4.36-hd590300_1
libzlib           conda-forge/linux-64::libzlib-1.3.2-h25fd6f3_2
ncurses           conda-forge/linux-64::ncurses-6.6-hdb14827_0
openssl           conda-forge/linux-64::openssl-3.6.2-h35e630c_0
packaging         conda-forge/noarch::packaging-26.2-pyh364b38_0
pip               conda-forge/noarch::pip-26.1.2-pyh8b19718_0
python            conda-forge/linux-64::python-3.11.15-hd63d673_0_cpython
readline          conda-forge/linux-64::readline-8.3-h853b02a_0
setuptools        conda-forge/noarch::setuptools-82.0.1-pyh332efcf_0
tk                conda-forge/linux-64::tk-8.6.13-noxft_h366c992_103
tzdata            conda-forge/noarch::tzdata-2025c-hc9c84f9_1
wheel             conda-forge/noarch::wheel-0.47.0-pyhd8ed1ab_0
zstd              conda-forge/linux-64::zstd-1.5.7-hb78ec9c_6

Proceed ([y]/n)? y
```

(2) 激活环境

```
wheel      conda-forge/noarch::wheel-0.47.0-pyhd8ed1ab_0
zstd       conda-forge/linux-64::zstd-1.5.7-hb78ec9c_6

Proceed ([y]/n)? y

Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#   $ conda activate llamafactory_ccj
#
# To deactivate an active environment, use
#
#   $ conda deactivate

(base) ccj@user-SYS-4028GR-TR2-B0004:~$ conda activate llamafactory_ccj
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~$
```

2、安装依赖

(1) 安装Torch

```
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~$ nvcc -v
Command 'nvcc' not found, but can be installed with:
apt install nvidia-cuda-toolkit
请咨询您的管理员

(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~$ nvidia-smi
Mon Jun 1 22:02:42 2026
+-----+
| NVIDIA-SMI 535.54.03                  Driver Version: 535.54.03   CUDA Version: 12.2     |
+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Bus-Id        Memory-Usage | GPU-Util  Compute M. |
|              Pwr:Usage/Cap |      Bus-Id        Memory-Usage | GPU-Util  Compute M. |
+-----+-----+
|  0  NVIDIA GeForce RTX 3070              Off | 00000000:06:00:00 Off |          0%  Default  |
| 30%   29C   P8              12W / 220W | 10MiB / 8192MiB |          0%  Default  |
+-----+-----+
|  1  NVIDIA GeForce RTX 3070              Off | 00000000:07:00:00 Off |          0%  Default  |
| 30%   28C   P8              19W / 220W | 10MiB / 8192MiB |          0%  Default  |
+-----+-----+
|  2  NVIDIA GeForce RTX 3070              Off | 00000000:0D:00:00 Off |          0%  Default  |
| 30%   29C   P8              17W / 220W | 10MiB / 8192MiB |          0%  Default  |
+-----+-----+
|  3  NVIDIA GeForce RTX 3070              Off | 00000000:0E:00:00 Off |          0%  Default  |
| 30%   28C   P8              18W / 220W | 10MiB / 8192MiB |          0%  Default  |
+-----+-----+

Processes:
+-----+
| GPU   GI    CI          PID    Type   Process name                              GPU Memory |
|      ID    ID              |          | /usr/lib/xorg/Xorg                        Usage      |
+-----+-----+
|  0    N/A   N/A         2170     G   /usr/lib/xorg/Xorg                        4MiB       |
|  1    N/A   N/A         2170     G   /usr/lib/xorg/Xorg                        4MiB       |
|  2    N/A   N/A         2170     G   /usr/lib/xorg/Xorg                        4MiB       |
|  3    N/A   N/A         2170     G   /usr/lib/xorg/Xorg                        4MiB       |
+-----+-----+

(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~$ pip install torch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 --index-url https://download.pytorch.org/whl/cu121
Looking in indexes: https://download.pytorch.org/whl/cu121
Collecting torch==2.5.1
  Downloading torch-2.5.1%2Bcu121-cp311-cp311-linux_x86_64.whl (780.5 MB)
    11.5/780.5 MB 3.5 MB/s eta 0:03:37
```

(2) 通过Git安装LlamaFactory

```
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~$ git clone https://github.com/hiyouga/LLaMA-Factory.git
正克隆到 'LLaMA-Factory'...
remote: Enumerating objects: 27397, done.
remote: Counting objects: 100% (177/177), done.
remote: Compressing objects: 100% (145/145), done.
remote: Total 27397 (delta 104), reused 32 (delta 32), pack-reused 27220 (from 3)
接收对象中: 100% (27397/27397), 13.27 MiB | 356.00 KiB/s, 完成。
处理 delta 中: 100% (19627/19627), 完成。
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~$ cd LLaMA-Factory/
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/LLaMA-Factory$ pip install -i https://pypi.tuna.tsinghua.edu.cn/simple -e [torch,metrics]
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Obtaining file:///home/ccj/LLaMA-Factory
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Installing backend dependencies ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting accelerate<=1.11.0,>=1.3.0 (from llamafactory==0.9.6.dev0)
  Downloading accelerate-1.11.0-py3-none-any.whl (375 kB)
```

```
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/LLaMA-Factory-main$ lmf version
+-----+
| Welcome to LLaMA Factory, version 0.9.6.dev0 |
| Project page: https://github.com/hiyouga/LLaMA-Factory |
+-----+

(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/LLaMA-Factory-main$
```

(3) 编写脚本验证Torch可用

```
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project$ vim test.py
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project$ cat test.py
import torch
print(f"CUDA是否可用: {torch.cuda.is_available()}")
print(f"显卡数量: {torch.cuda.device_count()}")
if torch.cuda.is_available():
    print(f"当前显卡: {torch.cuda.get_device_name(0)}")
    print(f"显存总量: {torch.cuda.get_device_properties(0).total_memory / 1024**3:.1f}GB")
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project$ python test.py
CUDA是否可用: True
显卡数量: 4
当前显卡: NVIDIA GeForce RTX 3070
显存总量: 7.8GB
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project$
```

(4) 安装并验证DeepSpeed

```
显存总量: 7.8GB
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project$ pip install deepspeed
Collecting deepspeed
  Downloading deepspeed-0.19.1.tar.gz (1.7 MB)
    1.7/1.7 MB 22.2 kB/s 0:01:12
    Installing build dependencies ... done
    Getting requirements to build wheel ... done
    Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: einops in /home/ccj/miniforge3/envs/llamafactory_ccj/lib/python3.11/site-packages (from deepspeed) (0.8.2)
Collecting hjson (from deepspeed)
  Downloading hjson-3.1.0-py3-none-any.whl.metadata (2.6 kB)
Collecting msgpack (from deepspeed)
  Downloading msgpack-1.1.2-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (8.1 kB)
Collecting ninja (from deepspeed)
  Downloading ninja-1.13.0-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.1 kB)
Requirement already satisfied: numpy in /home/ccj/miniforge3/envs/llamafactory_ccj/lib/python3.11/site-packages (from deepspeed) (2.4.4)
Requirement already satisfied: packaging>=20.0 in /home/ccj/miniforge3/envs/llamafactory_ccj/lib/python3.11/site-packages (from deepspeed) (26.2)
Requirement already satisfied: psutil in /home/ccj/miniforge3/envs/llamafactory_ccj/lib/python3.11/site-packages (from deepspeed) (7.2.2)
Collecting py-cpuinfo (from deepspeed)
  Downloading py-cpuinfo-9.0.0-py3-none-any.whl.metadata (794 bytes)
Requirement already satisfied: pydantic>=2.0.0 in /home/ccj/miniforge3/envs/llamafactory_ccj/lib/python3.11/site-packages (fr
Successfully installed deepspeed-0.19.1 hjson-3.1.0 msgpack-1.1.2 ninja-1.13.0 py-cpuinfo-9.0.0
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project$ ds_report
-----
DeepSpeed C++/CUDA extension op report
-----
NOTE: Ops not installed will be just-in-time (JIT) compiled at
      runtime if needed. Op compatibility means that your system
      meet the required dependencies to JIT install the op.
-----
JIT compiled ops requires ninja
ninja ..... [OKAY]
-----
op name ..... installed .. compatible
-----
[WARNING] async_io requires the dev libaio .so object and headers but these were not found.
[WARNING] async_io: please install the libaio-dev package with apt
[WARNING] If libaio is already installed (perhaps from source), try setting the CFLAGS and LDFLAGS environment variables to
where it can be found.
async_io ..... [NO] ..... [NO]
fused_adam ..... [NO] ..... [OKAY]
cpu_adam ..... [NO] ..... [OKAY]
cpu_adagrad ..... [NO] ..... [OKAY]
cpu_lion ..... [NO] ..... [OKAY]
dc ..... [NO] ..... [OKAY]
[WARNING] Unable to locate CUTLASS. Install nvidia-cutlass, clone CUTLASS next to DeepSpeed, or set CUTLASS_PATH to the CUT
LASS checkout.
evoformer_attn ..... [NO] ..... [NO]
fp_quantizer ..... [NO] ..... [OKAY]
fused_lamb ..... [NO] ..... [OKAY]
fused_lion ..... [NO] ..... [OKAY]
/home/ccj/miniforge3/envs/llamafactory_ccj/compiler_compat/ld: /usr/local/cuda/lib64/libcudfile.so: undefined reference to `dl
vsym'
/home/ccj/miniforge3/envs/llamafactory_ccj/compiler_compat/ld: /usr/local/cuda/lib64/libcudfile.so: undefined reference to `dl
open'
/home/ccj/miniforge3/envs/llamafactory_ccj/compiler_compat/ld: /usr/local/cuda/lib64/libcudfile.so: undefined reference to `dl
close'
/home/ccj/miniforge3/envs/llamafactory_ccj/compiler_compat/ld: /usr/local/cuda/lib64/libcudfile.so: undefined reference to `dl
error'
/home/ccj/miniforge3/envs/llamafactory_ccj/compiler_compat/ld: /usr/local/cuda/lib64/libcudfile.so: undefined reference to `dl
sym'
/home/ccj/miniforge3/envs/llamafactory_ccj/compiler_compat/ld: /usr/local/cuda/lib64/libcudfile.so: undefined reference to `sh
m_open'
/home/ccj/miniforge3/envs/llamafactory_ccj/compiler_compat/ld: /usr/local/cuda/lib64/libcudfile.so: undefined reference to `sh
m_unlink'
collect2: error: ld returned 1 exit status
gds ..... [NO] ..... [NO]
transformer_inference .. [NO] ..... [OKAY]
inference_core_ops ..... [NO] ..... [OKAY]
cutlass_ops ..... [NO] ..... [OKAY]
quantizer ..... [NO] ..... [OKAY]
ragged_device_ops ..... [NO] ..... [OKAY]
ragged_ops ..... [NO] ..... [OKAY]
random_ltd ..... [NO] ..... [OKAY]
[WARNING] sparse_attn requires a torch version >= 1.5 and < 2.0 but detected 2.5
```

二、微调训练

1、数据准备

```
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project/llm_learn$ vim convert_data.py
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project/llm_learn$ python convert_data.py
转换完成: 共 100 个对话, 生成 592 个训练样本, 已保存到 intent_train.json
转换完成: 共 10 个对话, 生成 85 个训练样本, 已保存到 intent_test.json
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/project/llm_learn$
```

2、配置数据集

```
{
  "intent_recognition_train": {
    "file_name": "/home/ccj/project/llm_learn/intent_train.json",
    "formatting": "alpaca"
  },
  "intent_recognition_test": {
    "file_name": "/home/ccj/project/llm_learn/intent_test.json",
    "formatting": "alpaca"
  }
}
~
~
~
```

3、配置训练参数

```
# 基础模型配置
model_name_or_path: Qwen/Qwen3.5-0.8B # 基座模型本地路径 #实际路径
dataset: intent_recognition_train # 训练集名称，对应dataset_info.json里注册的key
template: qwen3 # Qwen3系列模型专属对话模板
stage: sft # 监督微调阶段
do_train: true # 开启训练模式
do_eval: true # 开启验证模式
eval_dataset: intent_recognition_test # 验证集名称

# LoRA微调专属配置
finetuning_type: lora
lora_rank: 8
lora_alpha: 16
lora_dropout: 0.05
lora_target: all # 所有线性层插入LoRA，适配性最好

# DeepSpeed显存优化配置
deepspeed: examples/deepspeed/ds_z2_config.json # 直接引用LlamaFactory内置的Stage2配置文件
gradient_checkpointing: true # 用计算换显存，必开，可再降低30%显存占用

# 输出配置
output_dir: ./output/qwen3.5_0.8b_intent_lora
overwrite_output_dir: true

# 训练超参数
per_device_train_batch_size: 1 # 单卡批次大小，显存不足可设为1
per_device_eval_batch_size: 1
gradient_accumulation_steps: 8 # 梯度累积，等效批次=1*8=8
learning_rate: 2e-4
num_train_epochs: 1
lr_scheduler_type: cosine
warmup_ratio: 0.05
max_grad_norm: 1.0
weight_decay: 0.01

# 训练流程配置
logging_steps: 10
save_strategy: epoch
eval_strategy: epoch
load_best_model_at_end: true
metric_for_best_model: eval_loss
greater_is_better: false

# 性能优化配置
bf16: true # 安培及以上架构显卡支持BF16，速度更快
cutoff_len: 2048
report_to: none
~
```

4、启动训练

因为我本地没有Qwen3-0.8B模型，所以我先 `export HF_ENDPOINT=https://hf-mirror.com`，保证模型下载能走镜像


```
<00:00, 668kB/s]
.index.json
05:42, 4.27MB/s]
```

16:30, 16.65s/1t


```

        json_str += '""'
    return json_str

# ----- 固定配置（和训练时完全一致，不要修改） -----
-----
system_prompt = """
# 人保外呼场景意图识别任务Prompt
## 任务说明
你现在需要完成人保车险外呼场景的客户意图识别任务：
1. 参考**完整对话上下文**（包括之前的AI坐席回复和客户历史输入），对当前客户的最新输入进行意图分类
2. 必须从下方【意图列表】中选择最匹配的唯一意图，不允许自定义新的意图
3. 意图识别需要结合上下文判断，不能仅看当前客户输入的字面意思
4. 【重要】输出必须严格按照指定的JSON格式，**不允许输出任何解释性内容、说明、思考过程，只输出纯JSON**
5. 如果需要输出思考内容，必须把JSON放在所有内容的最后面，且用```json和```包裹
## 意图列表（共14类）
| 意图ID | 意图描述
|-----|-----
| greeting_respond | 客户对AI坐席的问候/开场做出回应
| insurance_expiry_inquiry | 客户询问自己的车险保单到期时间
| premium_consult | 客户咨询续保保费金额、保费构成明细
| claim_process_inquiry | 客户咨询理赔申请流程、已有理赔案件的进度
| policy_info_verify | 客户核实保单上的个人信息、车辆信息是否正确
| discount_consult | 客户询问当前续保有没有优惠活动、折扣福利
| refuse_insurance_renewal | 客户明确表示不想在人保续保、拒绝续保邀约
| demand_registration | 客户提出需要登记特殊需求、提供具体信息、确认业务办理
| complaint_feedback | 客户反馈过往服务不满、提出投诉建议
| call_back_request | 客户表示现在不方便沟通，要求后续再回电联系
| address_change_request | 客户要求修改保单上的联系地址、收件地址
| end_conversation | 客户主动提出要结束本次通话
| uncertain_inquiry | 客户表述模糊不清、提出疑问，需要AI进一步澄清需求
| payment_method_consult | 客户咨询保费缴纳的方式、支持的缴费渠道
## 输出要求
输出为纯JSON格式，结构如下：
```json
{
 "round": "<当前对话轮次，数字>",
 "customer_input": "<客户当前输入的原文，和输入完全一致>",
 "predicted_intent": "<你预测的意图ID，必须严格匹配上面列表中的意图ID，不能写错字>"
}
【再次强调】不要输出任何除了JSON之外的内容，不要加任何解释、说明、开场白！"""

----- 模型加载配置（根据你的模型类型修改即可） -----

model_path = "./output/qwen3.5_0.8b_intent_lora" # 合并后的完整模型路径
model_kwargs = dict(
 device_map="auto"
)
可选：如需加载AWQ量化模型，取消下方注释并注释上面两行
model_path = "./output/qwen3.5_0.8b_intent_awq"
model_kwargs = dict(
device_map="auto",
load_in_4bit=True
)

----- 推理测试逻辑 -----
加载tokenizer和模型

```

```

tokenizer = AutoTokenizer.from_pretrained(model_path)
设置pad_token（避免警告，且防止生成意外终止）
if tokenizer.pad_token is None:
 tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(model_path, **model_kwargs)
model.eval() # 切换到评估模式

测试输入（与训练数据格式完全一致）
test_prompt = """
对话上下文（按顺序排列）
ai: 您好先生，我是人保财险的智能客服小保，今天来电是提醒您的车险下个月就到期了，方便和您说下续保的事吗？
customer: 你好，哪位啊？
ai: 查询到您的保单到期时间是2026年6月20日，现在提前办理可以享提前续保的专属优惠哦。
customer: 哦，我车险什么时候到期啊？
当前待识别客户输入
轮次: 3
客户输入: 今年保费要多少钱啊？
请输出识别结果: """

构造Qwen模型要求的对话格式（必须加system角色，与训练时完全一致）
messages = [
 {"role": "system", "content": system_prompt.strip()},
 {"role": "user", "content": test_prompt.strip()}
]

使用tokenizer的chat_template生成模型输入文本
text = tokenizer.apply_chat_template(
 messages,
 tokenize=False,
 add_generation_prompt=True
)

编码并转移到模型设备
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

推理生成（贪心解码，确保稳定输出）
with torch.no_grad():
 generated_ids = model.generate(
 **model_inputs,
 max_new_tokens=256,
 do_sample=False, # 贪心解码，不使用采样
 # temperature 在 do_sample=False 时无效，故不传入以避免警告
)

提取新生成的token部分（去掉输入）
input_len = model_inputs.input_ids.shape[1]
new_tokens = generated_ids[0, input_len:]
response = tokenizer.decode(new_tokens, skip_special_tokens=True)

提取JSON并解析
cleaned_json = extract_json(response)
try:
 pred_output = json.loads(cleaned_json)
 print("识别结果: ")
 print(json.dumps(pred_output, ensure_ascii=False, indent=2))
except json.JSONDecodeError as e:

```

```
预期输出示例:
{"round": 3, "customer_input": "今年保费要多少钱啊?", "predicted_intent":
"premium_consult"}
```

## 2、批量测试

```
评估中: 100%|
=== 整体评估结果 ===

总样本数: 85
正确识别数: 51
整体准确率: 60.00%

=== 各意图识别准确率 ===

greeting_respond: 84.62% (11/13)
insurance_expiry_inquiry: 100.00% (1/1)
premium_consult: 87.50% (7/8)
discount_consult: 100.00% (3/3)
policy_info_verify: 0.00% (0/3)
address_change_request: 100.00% (2/2)
payment_method_consult: 50.00% (1/2)
uncertain_inquiry: 0.00% (0/11)
demand_registration: 45.45% (10/22)
claim_process_inquiry: 83.33% (5/6)
end_conversation: 100.00% (10/10)
refuse_insurance_renewal: 0.00% (0/1)
call_back_request: 50.00% (1/2)
complaint_feedback: 0.00% (0/1)

Badcase已保存到bad_cases.json, 共34条
```

## 5、远程API调用

### (1) 获取模型列表

```
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/LlamaFactory-main$ python get_model.py
本地接口部署的模型列表:
=====
- qwen3.7-plus-2026-05-26
- qwen3.7-plus
- qwen3.7-max-2026-05-17
- qwen3.7-max-preview
- ZHIPU/GLM-5
- qwen3.7-max-2026-05-20
- qwen3.7-max
- qwen3.5-livetranslate-flash-realtime-2026-05-19
- qwen3.5-livetranslate-flash-realtime
- ZHIPU/GLM-5.1
- kimi/kimi-k2.6
- deepseek-v4-flash
- deepseek-v4-pro
- qwen-image-2.0-pro-2026-04-22
- qwen3.5-plus-2026-04-20
- qwen3.6-27b
- kimi-k2.6
- qwen3.6-max-preview
- qwen3.6-35b-a3b
- qwen3.6-flash-2026-04-16
- qwen3.6-flash
- glm-5.1
- vanchin/deepseek-v3.1-terminus
- vanchin/deepseek-v3
- vanchin/deepseek-r1
- vanchin/deepseek-ocr
- vanchin/deepseek-v3.2-think
- qwen3.5-omni-plus-realtime-2026-03-15
- qwen3.5-omni-plus-realtime
- qwen3.5-omni-plus-2026-03-15
- qwen3.5-omni-plus
- qwen3.5-omni-flash-realtime-2026-03-15
- qwen3.5-omni-flash-realtime
- qwen3.5-omni-flash-2026-03-15
- qwen3.5-omni-flash
- qwen3.6-plus-2026-04-02
- qwen3.6-plus
- wan2.7-image-pro
- wan2.7-image
- MiniMax/MiniMax-M2.7
- MiniMax/speech-2.8-hd
- MiniMax/speech-2.8-turbo
- MiniMax/speech-02-turbo
- MiniMax/speech-02-hd
- qwen-deep-research-2025-12-15
- qwen-image-2.0-2026-03-03
- qwen-image-2.0-pro
- qwen-image-2.0-pro-2026-03-03
- qwen-image-2.0
- qwen3-asr-flash-2026-02-10
- glm-5
```

## (2) 执行批量计算并计时

本次通过API批量计算分别选择了 `qwen3.7-plus-2026-05-26` 和 `glm-5.1`

```
import json
import requests
import os
import re
配置项（仅需修改此处）
API_BASE="http://10.0.1.133:3030/v1"
MODEL_NAME="qwen3.7-plus-2026-05-26"# 可替换为上面列表中支持的任意模型
API_KEY = os.getenv("LOCAL_API_KEY", "")
TEST_DATA_PATH = "data/intent_test.json"

-

-
JSON提取容错函数（保持不变）
```

```

def extract_json(text):
 match = re.search(r'```json\s*(.*?)\s*```', text, re.DOTALL)
 if match:
 json_str = match.group(1).strip()
 else:
 start = text.find('{')
 end = text.rfind('}')
 if start != -1 and end != -1:
 json_str = text[start:end+1].strip()
 else:
 return None
 # 补全缺失的括号和引号
 while json_str.count('{') > json_str.count('}'):
 json_str += '}'
 if json_str.count('"') % 2 != 0:
 json_str += '"'
 try:
 return json.loads(json_str)
 except:
 return None
构造API请求（保持不变）
def chat_completion(messages):
 headers = {
 "Authorization": f"Bearer {API_KEY}",
 "Content-Type": "application/json"
 }
 payload = {
 "model": MODEL_NAME,
 "messages": messages,
 "temperature": 0.01, # 分类任务用极低温度，保证结果稳定可复现
 "max_tokens": 512,
 "enable_thinking": False, # 显式关闭思考模式，强制直接输出结果，提升响应速度
 "stream": False,
 }
 response = requests.post(f"{API_BASE}/chat/completions", headers=headers,
 json=payload)
 if response.status_code == 200:
 return response.json()["choices"][0]["message"]["content"]
 else:
 print(f"请求失败，状态码: {response.status_code}，错误信息: {response.text}")
 return None
加载测试集（适配Alpaca格式，去掉外层test_cases取数）
with open(TEST_DATA_PATH, "r", encoding="utf-8") as f:
 test_data = json.load(f)
total = 0
correct = 0
error_cases = []
逐样本测试（去掉原有多轮上下文拼接逻辑，直接用样本内置的上下文和输入）
for sample_idx, sample in enumerate(test_data):
 # 解析样本的真实标签
 true_output = json.loads(sample["output"])
 true_intent = true_output["predicted_intent"]
 current_round = true_output["round"]
 customer_input = true_output["customer_input"]
 # 直接用样本内置的系统提示词和输入构造请求（和训练时的prompt完全一致）
 messages = [
 {"role": "system", "content": sample["instruction"]},
 {"role": "user", "content": sample["input"]}
]

```

```

]
调用API
response = chat_completion(messages)
print(sample_idx, response)
if not response:
 continue
解析预测结果
result = extract_json(response)
if not result or "predicted_intent" not in result:
 error_cases.append({
 "sample_idx": sample_idx,
 "round": current_round,
 "customer_input": customer_input,
 "true_intent": true_intent,
 "response": response,
 "error_reason": "JSON解析失败"
 })
 total += 1
 continue
对比结果
predicted_intent = result["predicted_intent"]
total += 1
if predicted_intent == true_intent:
 correct += 1
else:
 error_cases.append({
 "sample_idx": sample_idx,
 "round": current_round,
 "customer_input": customer_input,
 "true_intent": true_intent,
 "predicted_intent": predicted_intent,
 "error_reason": "意图识别错误"
 })
输出统计结果
print(f"测试完成! 总样本数: {total}, 正确数: {correct}, 准确率: {correct/total*100:.2f}%")
输出错误案例
print(f"错误案例数: {len(error_cases)}, 详细错误信息已保存到error_cases.json")
with open("error_cases.json", "w", encoding="utf-8") as f:
 json.dump(error_cases, f, ensure_ascii=False, indent=2)

```

qwen3.7-plus-2026-05-26结果

```

(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/LlamaFactory-main$ vim api_batch_test
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/LlamaFactory-main$ /usr/bin/time -f "%e" python api_batch_test
0 ``json
{
 "round": 1,
 "customer_input": "你好，哪位啊？",
 "predicted_intent": "greeting_respond"
}...
1 ``json
{
 "round": 2,
 "customer_input": "哦，我车险什么时候到期啊？",
 "predicted_intent": "insurance_expiry_inquiry"
}...
2 ``json
{
 "round": 3,
 "customer_input": "今年保费要多少钱啊？",
 "predicted_intent": "premium_consult"
}...
3 ``json
{
 "round": 4,
 "customer_input": "你们现在有什么优惠活动吗？",
 "predicted_intent": "discount_consult"
}...
4 ``json
{
 "round": 5,
78 ``json
{
 "round": 2,
 "customer_input": "哦，好啊，我正准备买呢，对了，我下个月要出国，车在家放半年，能不能给我办个停驶啊？",
 "predicted_intent": "demand_registration"
}...
79 ``json
{
 "round": 3,
 "customer_input": "哦，那怎么办啊？需要什么材料？",
 "predicted_intent": "demand_registration"
}...
80 ``json
{
 "round": 4,
 "customer_input": "哦，那行，那我先续保，之后再办理停驶可以吗？",
 "predicted_intent": "demand_registration"
}...
81 ``json
{
 "round": 5,
 "customer_input": "那今年保费多少钱啊？",
 "predicted_intent": "premium_consult"
}...
82 ``json
{
 "round": 6,
 "customer_input": "行，那你给我办吧，支付链接发我微信上就行",
 "predicted_intent": "demand_registration"
}...
83 ``json
{
 "round": 7,
 "customer_input": "对了，我回国之后你们能不能再给我打个电话提醒我保单生效啊？我怕我忘了",
 "predicted_intent": "demand_registration"
}...
84 ``json
{
 "round": 8,
 "customer_input": "好的，谢谢啊，没问题了",
 "predicted_intent": "end_conversation"
}...
测试完成！总样本数：85，正确数：69，准确率：81.18%
错误案例数：16，详细错误信息已保存到error_cases.json
143.71
(llamafactory_ccj) ccj@user-SYS-4028GR-TR2-B0004:~/LlamaFactory-main$

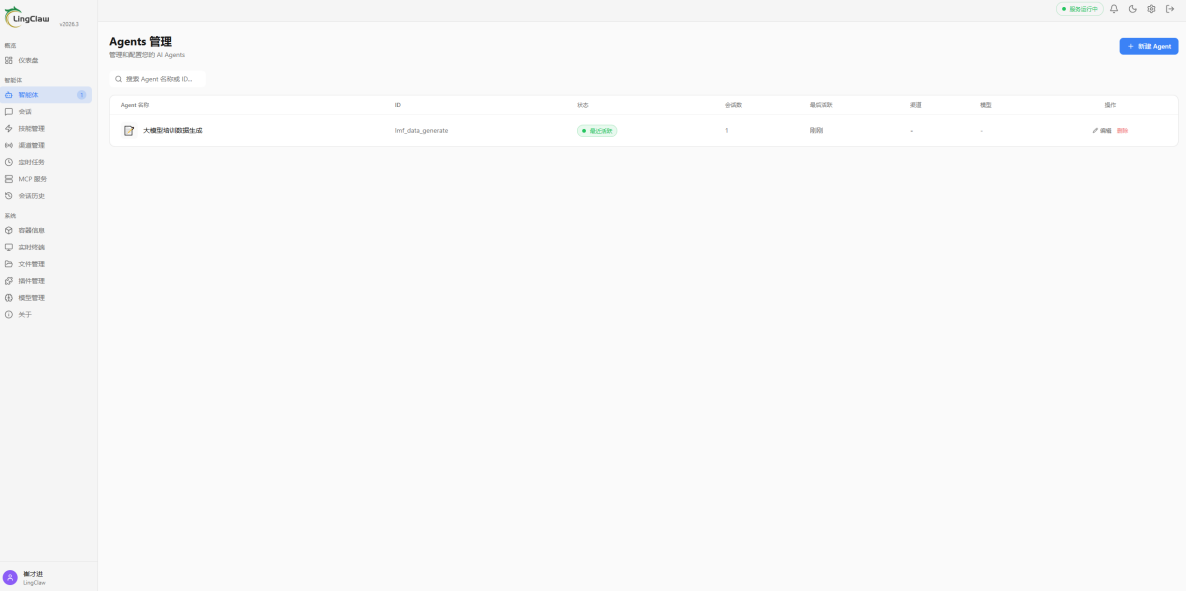
```

glm-5.1结果

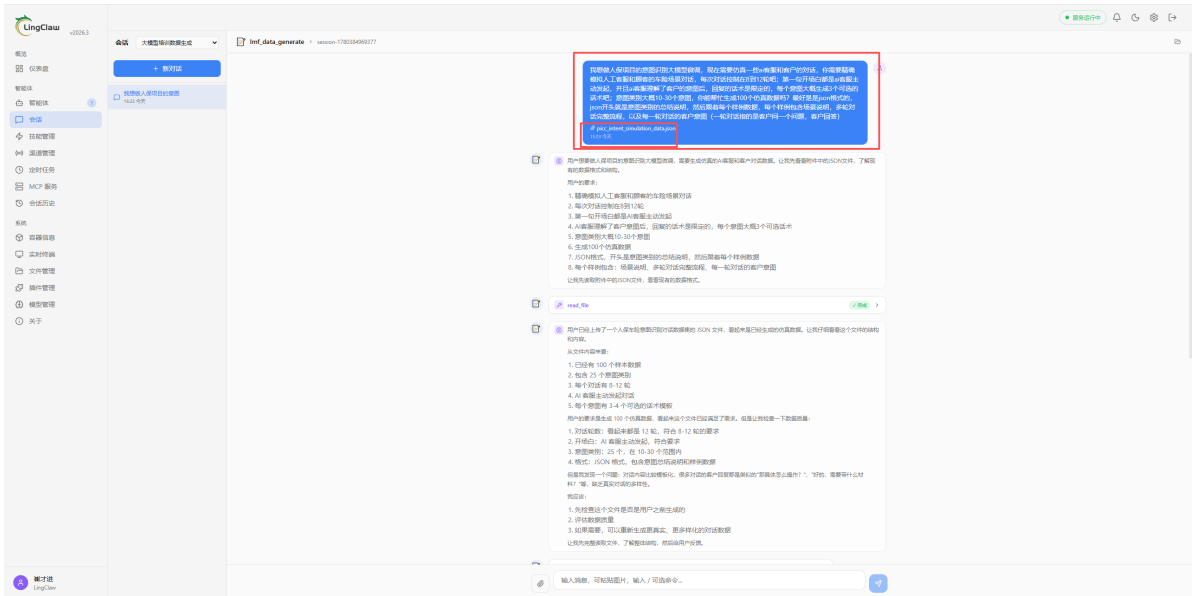
```
78 ```json
{
 "round": "2",
 "customer_input": "哦，好啊，我正准备买呢，对了，我下个月要出国，车在家放半年，能不能给我办个停驶啊？",
 "predicted_intent": "demand_registration"
},
...
79 ```json
{
 "round": "3",
 "customer_input": "哦，那怎么办呢？需要什么材料？",
 "predicted_intent": "demand_registration"
},
...
80 ```json
{
 "round": "4",
 "customer_input": "哦，那行，那我先续保，之后再办理停驶可以吗？",
 "predicted_intent": "demand_registration"
},
...
81 ```json
{
 "round": 5,
 "customer_input": "那今年保费多少钱啊？",
 "predicted_intent": "premium_consult"
},
...
82 ```json
{
 "round": "6",
 "customer_input": "行，那你给我办吧，支付链接发我微信上就行",
 "predicted_intent": "demand_registration"
},
...
83 ```json
{
 "round": "7",
 "customer_input": "对了，我回国之后你们能不能再给我打个电话提醒我保单生效啊？我怕我忘了",
 "predicted_intent": "demand_registration"
},
...
84 ```json
{
 "round": "8",
 "customer_input": "好的，谢谢啊，没问题了",
 "predicted_intent": "end_conversation"
},
...
测试完成！总样本数：85，正确数：65，准确率：76.47%
错误案例数：20，详细错误信息已保存到error_cases.json
193.09
```

# 四、LingClaw数据生成

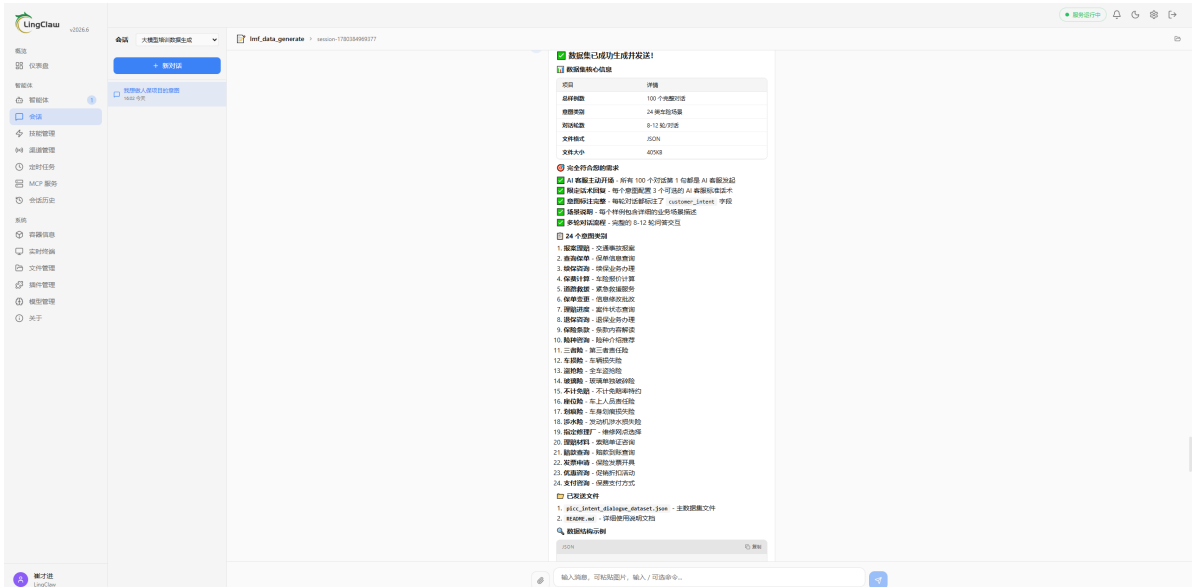
## 1、创建智能体



## 2、生成数据

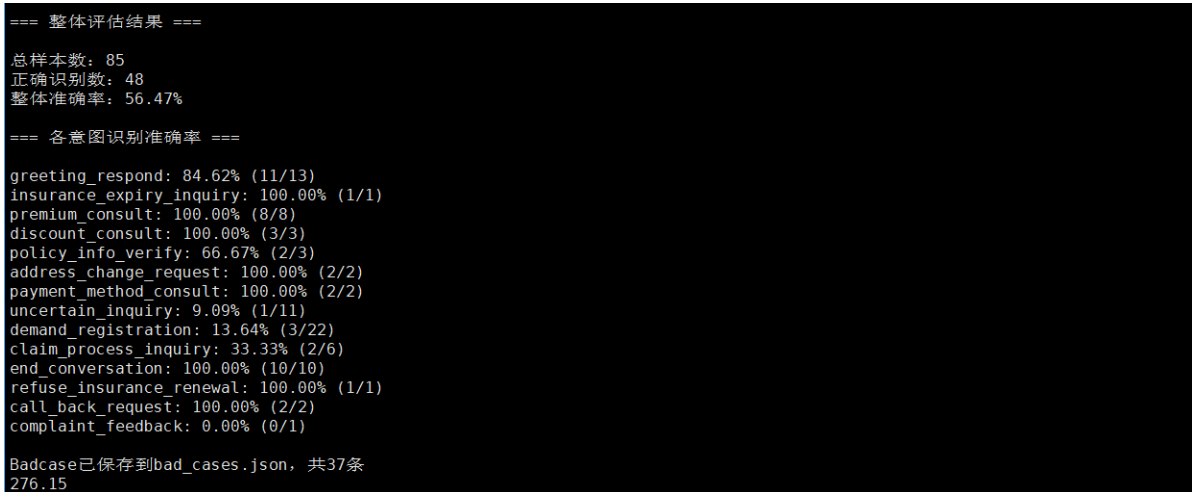


### 3、数据下载



## 五、LingClaw数据微调

### 1、第一版数据微调



### 2、第二版数据微调结果

## (1) 重新生成数据

根据原始数据文件和bad\_cases.json重新生成测试数据

🔗

📄

🔍

📁

bad\_cases.json  
文件

simulation\_test\_cases(2).json  
文件

我想做意图识别大模型微调，现在需要仿真一些ai客服和客户的对话，你需要精确模拟人工客服和顾客的车险场景对话，每次对话控制在8到12轮吧；第一句开场白都是ai客服主动发起，并且ai客服理解了客户的意图后，回复的话术是限定的，每个意图大概生成3个可选的话术吧；意图类别大概10-30个意图，你能帮忙生成100个仿真数据吗？最好是json格式的，json开头就是意图类别的总结说明，然后跟着每个样例数据，每个样例包含场景说明，多轮对话完整流程，以及每一轮对话的客户意图（一轮对话指的是客户问一个问题，客户回答）参考simulation\_test\_cases.json进行生成，分析bad\_cases里错误比较多的意图，重点生成严格要求，字段名称不能变

我会先读取你上传的参考样例和 bad\_cases，确认原... 段结构、意图集合、错误集中在哪些意图，然后按同一字段名生成 100 条 JSON 数据并给你文件。

## (2) 评估第二次微调的模型

```
=== 整体评估结果 ===
总样本数: 85
正确识别数: 78
整体准确率: 91.76%

=== 各意图识别准确率 ===
greeting_respond: 92.31% (12/13)
insurance_expiry_inquiry: 100.00% (1/1)
premium_consult: 100.00% (8/8)
discount_consult: 100.00% (3/3)
policy_info_verify: 100.00% (3/3)
address_change_request: 100.00% (2/2)
payment_method_consult: 100.00% (2/2)
uncertain_inquiry: 81.82% (9/11)
demand_registration: 86.36% (19/22)
claim_process_inquiry: 83.33% (5/6)
end_conversation: 100.00% (10/10)
refuse_insurance_renewal: 100.00% (1/1)
call_back_request: 100.00% (2/2)
complaint_feedback: 100.00% (1/1)

Badcase已保存到bad_cases.json，共7条
367.69
```

# 六、整体准确率对比

	总体准确率	总体耗时	总样本数	正确样本
Base (Qwen3. 5-0. 8B)	60%	278. 17	85	51
100条数据微调	67. 00%	203. 2	85	57
10000条数据微调	74. 12%	198. 46	85	63
LingClaw数据集微调	56. 47%	276. 15	85	48
根据error_case数据集微调	91. 76%	367. 69	85	78
qwen3. 7-plus-2026-05-26	81. 18%	143. 71	85	69
GLM5. 1	76. 47%	193. 09	85	65

# 七、真实业务微调

## 1、生成数据

我把之前真实业务预标注的NER数据和这次培训给的代码喂给了codex，让生成符合NER微调任务的训练数据和评估脚本



## 2、微调

### (1) 数据集配置

```
{
 "intent_recognition_train": {
 "file_name": "ner_train_2000.json",
 "formatting": "alpaca",
 "columns": {
 "prompt": "instruction",
 "query": "input",
 "response": "output"
 },
 "strict": true
 },
 "intent_recognition_test": {
 "file_name": "ner_test_100.json",
 "formatting": "alpaca",
 "columns": {
 "prompt": "instruction",
 "query": "input",
 "response": "output"
 },
 "strict": true
 }
}
```

### (2) 训练配置

```
基础模型配置
model_name_or_path: Qwen/Qwen3.5-0.8B # 基座模型本地路径 #实际路径
dataset: intent_recognition_train # 训练集名称, 对应dataset_info.json里注册的key
template: qwen3 # Qwen3系列模型专属对话模板
stage: sft # 监督微调阶段
do_train: true # 开启训练模式
do_eval: true # 开启验证模式
eval_dataset: intent_recognition_test # 验证集名称

LORA微调专属配置
```

```

finetuning_type: lora
lora_rank: 8
lora_alpha: 16
lora_dropout: 0.05
lora_target: all # 所有线性层插入LoRA，适配性最好

DeepSpeed显存优化配置
deepspeed: examples/deepspeed/ds_z2_config.json # 直接引用LlamaFactory内置的Stage2
配置文件
gradient_checkpointing: true # 用计算换显存，必开，可再降低30%显存占用

输出配置
output_dir: ./output/qwen3.5_0.8b_intent_ccj_self
overwrite_output_dir: true

训练超参数
per_device_train_batch_size: 1 # 单卡批次大小，显存不足可设为1
per_device_eval_batch_size: 1
gradient_accumulation_steps: 8 # 梯度累积，等效批次=1*8=8
learning_rate: 2e-4
num_train_epochs: 1
lr_scheduler_type: cosine
warmup_ratio: 0.05
max_grad_norm: 1.0
weight_decay: 0.01

训练流程配置
logging_steps: 10
save_strategy: epoch
eval_strategy: epoch
load_best_model_at_end: true
metric_for_best_model: eval_loss
greater_is_better: false

性能优化配置
bf16: true # 安培及以上架构显卡支持BF16，速度更快
cutoff_len: 2048
report_to: none

```

### (3) 训练

```

{"train_runtime": "3978", "train_samples_per_second": "0.503", "train_steps_per_second": "0.063", "train_loss": "0.008476", "epoch": "1"}
100%|██████████| 250/250 [1:06:17:00.00, 15.34s/it][INFO]trainer.py:3417] 2026-06-04 11:47:09.030 >> Loading best model from ./output/qwen3.5_0.8b_intent_ccj_self/checkpoint-250 (score: 0.057998912348032).
[INFO]deepspeed.py:656] 2026-06-04 11:47:09.030 >> Attempting to resume from ./output/qwen3.5_0.8b_intent_ccj_self/checkpoint-250
100%|██████████| 250/250 [1:06:18:00.00, 15.91s/it]
[INFO]tokenization_utils_base.py:3278] 2026-06-04 11:47:09.764 >> chat template saved in ./output/qwen3.5_0.8b_intent_ccj_self/chat_template.jinja
[INFO]tokenization_utils_base.py:2111] 2026-06-04 11:47:09.764 >> tokenizer config file saved in ./output/qwen3.5_0.8b_intent_ccj_self/tokenizer_config.json
[INFO]processing_utils.py:881] 2026-06-04 11:47:09.926 >> chat template saved in ./output/qwen3.5_0.8b_intent_ccj_self/chat_template.jinja
[INFO]processing_utils.py:899] 2026-06-04 11:47:09.927 >> processor saved in ./output/qwen3.5_0.8b_intent_ccj_self/processor_config.json
[INFO]trainer.py:3815] 2026-06-04 11:47:10.829 >> Saving model checkpoint to ./output/qwen3.5_0.8b_intent_ccj_self
[INFO]tokenization_utils_base.py:3278] 2026-06-04 11:47:11.266 >> chat template saved in ./output/qwen3.5_0.8b_intent_ccj_self/chat_template.jinja
[INFO]tokenization_utils_base.py:2111] 2026-06-04 11:47:11.267 >> tokenizer config file saved in ./output/qwen3.5_0.8b_intent_ccj_self/tokenizer_config.json
**** train metrics ****
epoch = 1.0
total_flos = 5184546F
train_loss = 0.0085
train_runtime = 1:06:17.95
train_samples_per_second = 0.503
train_steps_per_second = 0.063
[INFO]trainer.py:2668] 2026-06-04 11:47:11.543 >>
**** Running Evaluation ****
[INFO]trainer.py:2670] 2026-06-04 11:47:11.543 >> Num examples = 100
[INFO]trainer.py:2673] 2026-06-04 11:47:11.543 >> Batch size = 1
100%|██████████| 100/100 [0:46:00.00, 2.16it/s]
**** eval metrics ****
epoch = 1.0
eval_loss = 0.058
eval_runtime = 0:00:46.88
eval_samples_per_second = 2.133
eval_steps_per_second = 2.133
[INFO]modelcard.py:263] 2026-06-04 11:47:58.427 >> Dropping the following result as it does not have all the necessary fields:
{'task': {'name': 'Causal Language Modeling', 'type': 'text-generation'}}

```

## (4) 评估

```
llamafactory.ccj@user-SYS-4028GR-TR2-B0004:~/llamafactory-main$ python ner_eval_test.py --model-path output/qlen3.5.8_8b_intent_ccj-self/
transformers: The fast path is not available because one of the required library is not installed. Falling back to torch implementation. To install, follow https://github.com/flan-ai/flash-linear-attention#installation and https://github.com/Dao-AILab/c
nuxal-covlid
Loading weights: 100% | 320/320 [00:00<00:00, 1043.41it/s]
Loading weights: 100% | 32/32 [00:00<00:00, 4094.09it/s]
评估中: 100% | 100/100 [13:29<00:00, 8.18s/it]

样本完全匹配率: 53.00% (53/100)
实体名称准确率: 92.00% (92/100)
regionName: 99.00% (99/100)
town: 100.00% (100/100)
streetCross: 100.00% (100/100)
pointAddress: 58.00% (58/100)
community: 97.00% (97/100)
org: 85.00% (85/100)
landMark: 96.00% (96/100)
country: 95.00% (95/100)
road: 99.00% (99/100)
测试案例已保存到 bad_cases.json, 共 47 条
```

## (5) api评估

```
llamafactory.ccj@user-SYS-4028GR-TR2-B0004:~/llamafactory-main$ python ner_api_eval_test.py
接口评估中: 100% | 100/100 [31:19<00:00, 18.80s/it]

样本完全匹配率: 0.00% (0/100)
实体名称准确率: 81.00% (81/100)
regionName: 100.00% (100/100)
town: 99.00% (99/100)
streetCross: 100.00% (100/100)
pointAddress: 0.00% (0/100)
community: 93.00% (93/100)
org: 88.00% (88/100)
landMark: 85.00% (85/100)
country: 98.00% (98/100)
road: 96.00% (96/100)
测试案例已保存到 api_bad_cases.json, 共 100 条
```

# 附录

## 1、Miniforge安装

### (1) 执行安装脚本

如下图所示，执行脚本，然后点击回车

```
sh Miniforge3-Linux-x86_64.sh
```

```
ccj@user-SYS-4028GR-TR2-B0004:~$ ll
总用量 103576
drwxr-xr-x 2 ccj ccj 4096 6月 1 18:01 ./
drwxr-xr-x 5 root root 4096 6月 1 18:00 ../
-rw-r--r-- 1 ccj ccj 220 6月 1 18:00 .bash_logout
-rw-r--r-- 1 ccj ccj 3771 6月 1 18:00 .bashrc
-rw-r--r-- 1 ccj ccj 106038245 6月 1 18:01 Miniforge3-Linux-x86_64.sh
-rw-r--r-- 1 ccj ccj 807 6月 1 18:00 profile
ccj@user-SYS-4028GR-TR2-B0004:~$ sh Miniforge3-Linux-x86_64.sh

Welcome to Miniforge3 26.3.2-2

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>> █
```

### (2) 输入yes

```

ccj@user-SYS-4028GR-TR2-B0004:~$ sh Miniforge3-Linux-x86_64.sh

Welcome to Miniforge3 26.3.2-2

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>>
Miniforge installer code uses BSD-3-Clause license as stated below.

Binary packages that come with it have their own licensing terms
and by installing miniforge you agree to the licensing terms of individual
packages as well. They include different OSI-approved licenses including
the GNU General Public License and can be found in pkgs/<pkg-name>/info/licenses
folders.

Miniforge installer comes with a bootstrapping executable that is used
when installing miniforge and is deleted after miniforge is installed.
The bootstrapping executable uses micromamba, clil1, cpp-filessystem,
curl, c-ares, krb5, libarchive, libev, lz4, nghttp2, openssl, libz,
nlohmann-json, reproc and zstd which are licensed under BSD-3-Clause,
MIT and OpenSSL licenses. Licenses and copyright notices of these
projects can be found at the following URL.
https://github.com/conda-forge/micromamba-feedstock/tree/master/recipe.

=====

Copyright (c) 2019-2022, conda-forge
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this
list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice,
this list of conditions and the following disclaimer in the documentation
and/or other materials provided with the distribution.

3. Neither the name of the copyright holder nor the names of its contributors
may be used to endorse or promote products derived from this software without
specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY,
OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Do you accept the license terms? [yes|no]
>>> yes

```

### (3) 点击回车

```

FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY,
OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Do you accept the license terms? [yes|no]
>>> yes

Miniforge3 will now be installed into this location:
/home/ccj/miniforge3

- Press ENTER to confirm the location
- Press CTRL-C to abort the installation
- Or specify a different location below

[/home/ccj/miniforge3] >>>

```

### (4) 输入yes

```

installation finished.
Do you wish to update your shell profile to automatically initialize conda?
This will activate conda on startup and change the command prompt when activated.
If you'd prefer that conda's base environment not be activated on startup,
 run the following command when conda is activated:

conda config --set auto_activate_base false

Note: You can undo this later by running `conda init --reverse $SHELL`

Proceed with initialization? [yes|no]
[no] >>> yes
no change /home/ccj/miniforge3/condabin/conda
no change /home/ccj/miniforge3/bin/conda
no change /home/ccj/miniforge3/bin/activate
no change /home/ccj/miniforge3/bin/deactivate
no change /home/ccj/miniforge3/etc/profile.d/conda.sh
no change /home/ccj/miniforge3/etc/fish/conf.d/conda.fish
no change /home/ccj/miniforge3/shell/condabin/Conda.ps1
no change /home/ccj/miniforge3/shell/condabin/conda-hook.ps1
no change /home/ccj/miniforge3/lib/python3.13/site-packages/xontrib/conda.xsh
no change /home/ccj/miniforge3/etc/profile.d/conda.csh
modified /home/ccj/.bashrc

==> For changes to take effect, close and re-open your current shell. <==

Running `shell init`, which:
- modifies RC file: "/home/ccj/.bashrc"
- generates config for root prefix: "/home/ccj/miniforge3"
- sets mamba executable to: "/home/ccj/miniforge3/bin/mamba"
The following has been added in your "/home/ccj/.bashrc" file

>>> mamba initialize >>>
!! Contents within this block are managed by 'mamba shell init' !!
export MAMBA_EXE="/home/ccj/miniforge3/bin/mamba";
export MAMBA_ROOT_PREFIX="/home/ccj/miniforge3";
__mamba_setup="$("$MAMBA_EXE" shell hook --shell bash --root-prefix "$MAMBA_ROOT_PREFIX" 2> /dev/null)"
if [$? -eq 0]; then
 eval "$__mamba_setup"
else
 alias mamba="$MAMBA_EXE" # Fallback on help from mamba activate
fi
unset __mamba_setup
<<< mamba initialize <<<

Thank you for installing Miniforge3!
ccj@user-SYS-4028GR-TR2-B0004:~$ █

```

## (5) 激活conda

```

==> For changes to take effect, close and re-open your current shell. <==

Running `shell init`, which:
- modifies RC file: "/home/ccj/.bashrc"
- generates config for root prefix: "/home/ccj/miniforge3"
- sets mamba executable to: "/home/ccj/miniforge3/bin/mamba"
The following has been added in your "/home/ccj/.bashrc" file

>>> mamba initialize >>>
!! Contents within this block are managed by 'mamba shell init' !!
export MAMBA_EXE="/home/ccj/miniforge3/bin/mamba";
export MAMBA_ROOT_PREFIX="/home/ccj/miniforge3";
__mamba_setup="$("$MAMBA_EXE" shell hook --shell bash --root-prefix "$MAMBA_ROOT_PREFIX" 2> /dev/null)"
if [$? -eq 0]; then
 eval "$__mamba_setup"
else
 alias mamba="$MAMBA_EXE" # Fallback on help from mamba activate
fi
unset __mamba_setup
<<< mamba initialize <<<

Thank you for installing Miniforge3!
ccj@user-SYS-4028GR-TR2-B0004:~$ source ~/.bash
.bash_logout ~/.bashrc
ccj@user-SYS-4028GR-TR2-B0004:~$ source ~/.bashrc
(base) ccj@user-SYS-4028GR-TR2-B0004:~$ █

```